Project 1: Process and Resource Manager

1. Project Overview

This project develops a simple process and resource manager. In the most basic form, the manager supports the creation and management of the data structures to represent processes and resources. The manager also provides the basic operations invoked by processes to create and destroy processes, and to request and release resources. A manually invoked timeout function mimics the behavior of preemptive scheduling.

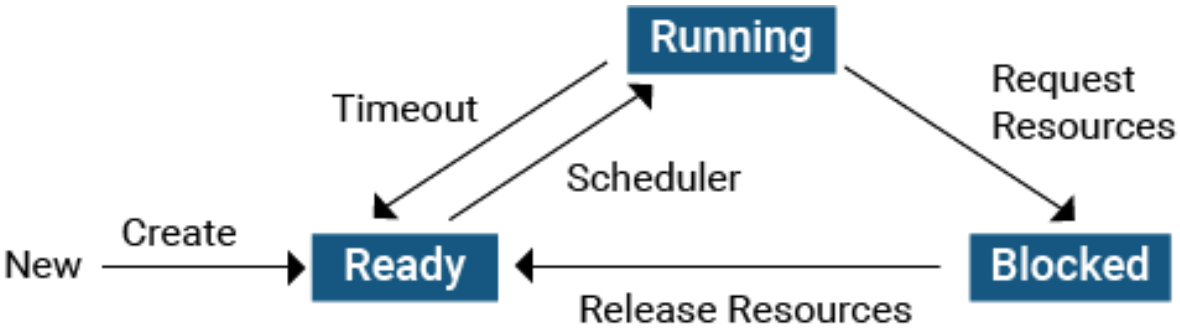
The manager can be tested using a presentation shell developed as part of the project. This allows the testing of the manager without running any actual processes and without having to access the physical CPU or other hardware components. Instead, the presentation shell plays the role of both the currently running process and the hardware by accepting commands typed in by the user and invoking the corresponding functions of the manager.

The basic manager can optionally be extended with additional functionalities: (1) processes have different priorities and are scheduled according to their priorities; (2) resources can have multiple identical units.

2. The Basic Process and Resource Manager

2.1 Process States

A process can be in one of three states: *ready*, *running*, *blocked*. The following diagram shows the possible state transitions:

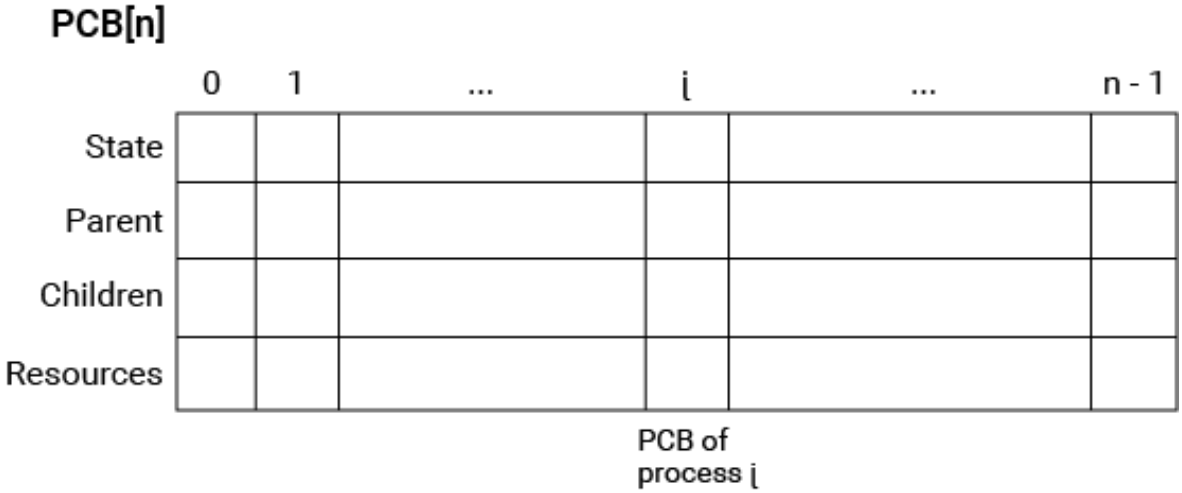


- A new process enters into the *ready* state.
- At some point in time, a *scheduler* function selects the process to run on the CPU and changes the state from *ready* to *running*.
- A *timeout* function can move the process from *running* back to *ready*.
- When a running process requests a currently unavailable resource, the state changes from *running* to *blocked*.
- When the resource is released and the process acquires the resource, the state changes from *blocked* to *ready*.

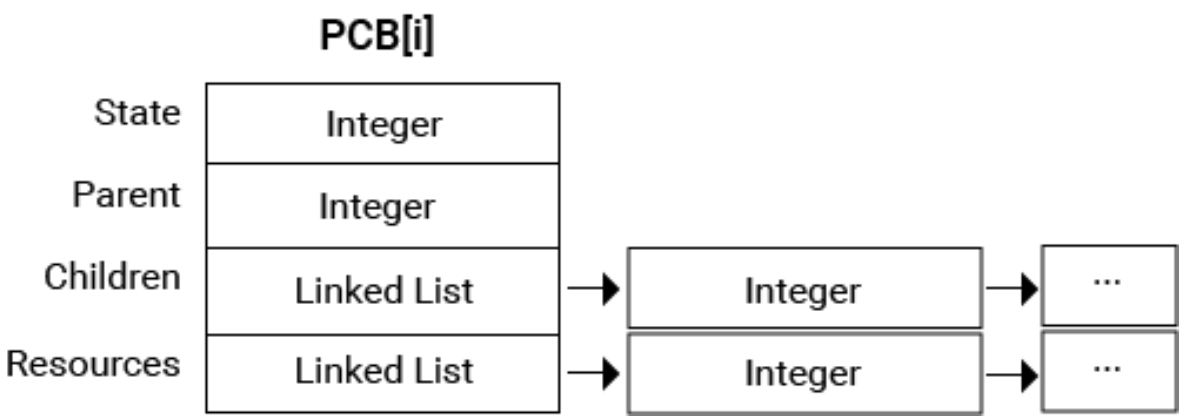
A process typically cycles through the states many times throughout its lifetime.

2.2 Representation of Processes

Each process is represented by a data structure called the *process control block (PCB)*. The PCBs are organized as a fixed-size array, PCB[n], where n is the maximum number of processes that can be created. Each process is uniquely identified by the PCB index:



For the basic version of the manager, each PCB only consists of the following fields, organized as a structure:



- *State* is the current state of the process i. The *running* state can be represented implicitly: the ready process at the head of the list of ready processes is always the running process. Thus, only the 2 states, *ready* and *blocked*, need to be implemented explicitly and can be encoded as integers or binary values (1 and 0).
- *Parent* is the index of the process that created process i.
- *Children* is the start of a linked list of processes that process i has created. Each element on the list contains the index of the corresponding child process.
- *Resources* is the start of a linked list of resources that the process i is currently holding. Each element on the list contains the index of the corresponding resource.

For scheduling purposes, the manager maintains all PCBs on one of several lists. Blocked processes are kept on waiting lists associated with the relevant resource, as explained in Section 2.4. All ready processes are kept on a *Ready List (RL)*. The basic version of the manager treats all processes as having the same priority and thus the RL is organized as a single linked list of PCB indices.

When the system is initialized, process 0 is created automatically and it becomes the first running process. All other processes are created and destroyed dynamically using the following operations.

2.3 Operations on Processes

The currently running process, i, can create a new child process, j, by invoking the *create* function:

```
create()
    allocate new PCB[j]
    state = ready
    insert j into children of i
    parent = i
    children = NULL
    resources = NULL
    insert j into RL
    display: "process j created"
```

The currently running process, i, can destroy one of its children, j, including all of j's descendants, by invoking the *destroy* function. The reason for destroying the entire subtree staring with process j is to avoid the creation of orphan processes, that is, processes without a parent. The process i can also destroy itself and all of its descendants by invoking destroy(i).

```
destroy(j)
    for all k in children of j destroy(k)
    remove j from parent's list
```

remove j from RL or waiting list

release all resources of j

free PCB of j

display: "n processes destroyed"

Use the following diagram to answer the 1.1 Knowledge Check Questions.

1.1 Knowledge Check

Your Status:

Your last completed attempt was on 04/12/2026

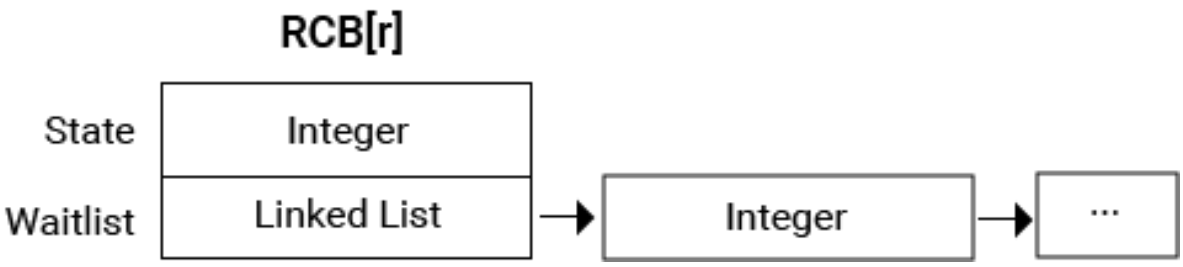
You may attempt this again. **BEGIN YOUR RETAKE (/LAUNCH-ASSESSMENT/17925471/2129806?TYPE=/WEBCOM-VIEW-PAGE/5/74302/74291%3FPAGE-ID%3D4829628)**.

2.4 Representation of Resources

The system supports a fixed set of resources that any process may request, acquire, and later release. When a process requests a resource that is currently unavailable, the process becomes blocked on that resource.

Each resource is represented by a data structure called the *resource control block (RCB)*. Analogous to the PCBs, the RCBs are organized as a fixed-size array, RCB[m], where m is the number of resources in the system. Each resource is uniquely identified by the RCB index.

For the basic version of the manager, each RCB consists only of the following fields, organized as a structure.



- *State* is the current state of the resource r. The 2 possible values are *free* or *allocated*, which can be encoded as integers or binary values (1 and 0).
- *Waitlist* is the start of a linked list of processes blocked on the resource. Each element on the list contains the index of one of the blocked processes.

All resources are static and their RCBs are created automatically when the system is initialized.

2.5 Operations on Resources

The currently running process, i, may request any of the resources, r, at any time using the *request* function:

```
request(r)
  if state of r is free
    state of r = allocated
    insert r into list of resources of process i
    display: "resource r allocated"
  else
```

```
  state of i = blocked
  move i from RL to waitlist of r
  display: "process i blocked"
  scheduler()
```

The currently running process, i, may release any of the resources, r, it is holding using the *release* function:

```
release(r)
  remove r from resources list of process i
  if waitlist of r is empty
    state of r = free
  else
    move process j from the head of waitlist of r to RL
    state of j = ready
    insert r into resources list of process j
  display: "resource r released"
```

Use the following diagram to determine the values of the fields containing a question mark for 1.2 Knowledge Check Questions.

1.2 Knowledge Check

Your Status:

Your last completed attempt was on 04/13/2026

You may attempt this again. **BEGIN YOUR RETAKE (/LAUNCH-ASSESSMENT/17925473/2129806?TYPE=/WEBCOM-VIEW-PAGE/5/74302/74291%3FPAGE-ID%3D4829628)**.

2.6 Time-Sharing and the Scheduler

In the basic version of the manager, all processes have the same priority. Thus, the RL is a single linked list of PCB indices accessed in first in, first out (FIFO) order. The process at the head of the RL is the currently running process.

The system mimics preemptive scheduling by a function *timeout()*, which moves the process, i, currently at the head of the RL, to the end of the RL, and calls the scheduler to perform the context switch.

```
timeout()
    move process i from head of RL to end of RL
    scheduler()
```

The new process, j, now at the head of the RL becomes the running process. Repeatedly invoking timeout thus mimics the idea of time-sharing.

The task of the *scheduler* function is to perform the context switch from the currently running process i to the new process j. The *scheduler* must be called whenever process i blocks on a resource and is removed from the RL, and whenever the *timeout* function moves the process to the end of the RL.

In a real system, a context switch saves the CPU state of the running process i (contents of all registers) and loads the CPU state of the new process j. In this project, we do not have access to the physical CPU to save and restore its registers. Thus, the task of the *scheduler* is to only display on your terminal the index of the currently running process. At that point, the user terminal begins to play the role of the currently running process.

```
scheduler()
    find process i currently at the head of RL
    display: "process i running"
```

2.7 System Initialization and Error Handling

When the system starts, the following data structures must be created: PCB[n], RCB[m], RL

In addition to the manager functions described so far, an *init()* function must be implemented, which initializes all data structures:

- All PCB entries are initialized to free except PCB[0].
- PCB[0] is initialized to be a running process with no parent, no children, and no resources.
- All RCB entries are initialized to free.
- RL contains process 0

The purpose of *init* is to allow continuous testing of the system without having to repeatedly terminate and restart the program.

Each of the functions of the manager must also implement error checks to detect any illegal or unanticipated operations.

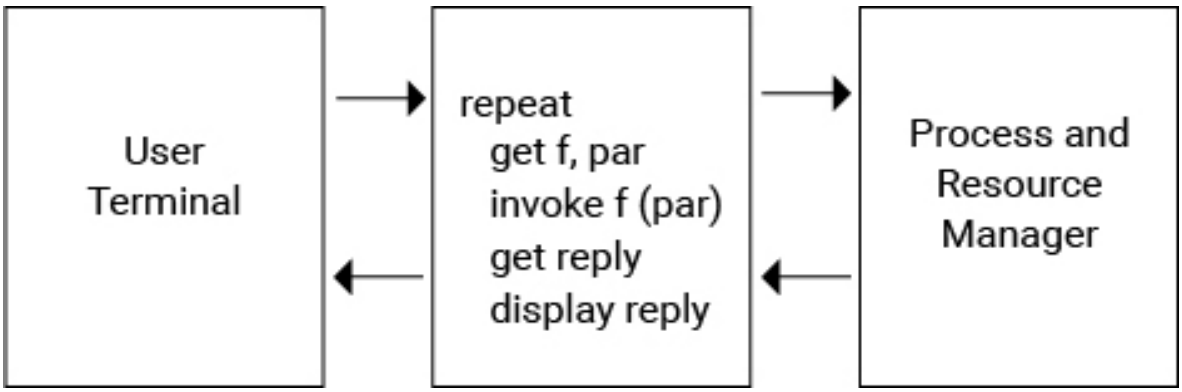
Examples of conditions that must be detected and reported include:

- Creating more than n processes
- Destroying a process that is not a child of the current process
- Requesting a nonexistent resource
- Requesting a resource the process is already holding
- Releasing a resource the process is not holding
- Process 0 should be prevented from requesting any resource to avoid the possibility of a deadlock where no process is on the RL.

3. The Presentation Shell

In this project, we do not have access to the physical CPU to save and restore its registers. We also do not execute the code of any process and have no access to a physical hardware interrupt to perform a timeout.

To test and demonstrate the functionality of the manager, we develop a presentation shell, which repeatedly accepts commands from the user terminal, invokes the corresponding function of the manager, and displays feedback messages on the screen to reflect the behavior of the system. The following diagram illustrates the basic structure:



Using the above organization, the user terminal represents the currently running process. That means, whenever a command is entered, it is interpreted as if the currently running process issued the command. The presentation shell invokes the corresponding function, f, of the manager, and passes to it any parameters (*par*) supplied from the terminal. The function invocation results in some changes to the PCBs, RCBs, and the RL. Each function then returns a message to the shell, indicating the success or failure or the last operation. The shell displays the message on the screen.

The user terminal also plays the role of the hardware, allowing the user to invoke the *timeout* function, that in a real system would be triggered by a hardware interrupt.

To simplify the interactions with the shell, we devise a syntax similar to the UNIX shell, where each line starts with a short command name, followed by optional parameters separated by blanks. The shell interprets each line and invokes the corresponding manager function. The following is a list of commands and the corresponding functions to be invoked:

Shell command	Function
cr	create()

Shell command	Function
de <i>	destroy(i)
rq <r>	request(r)
rl <r>	release(r)
to	timeout()
in	init()

The following sequence illustrates a typical interaction with the shell. Lines preceded by the asterisk (*) are responses displayed by the shell; lines preceded by the prompt sign (>) are commands typed in:

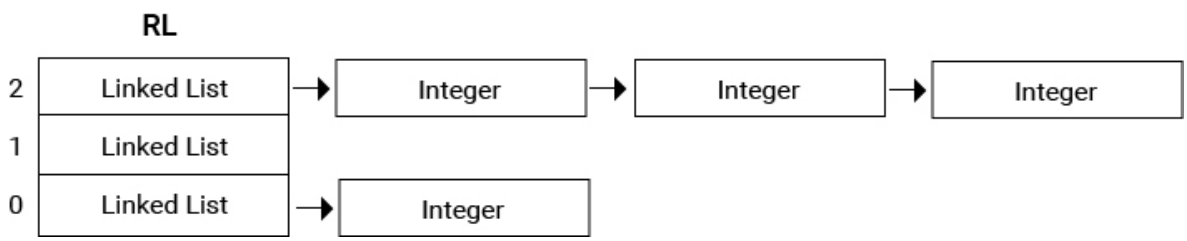
```
...
* process 5 running
> cr
* process 7 created
> to
* process 7 running
> rq 3
* resource 3 allocated
> to
* process 5 running
> de 7
* 1 process destroyed
> rl 2
* error
...
```

4. Extended Process and Resource Manager

The basic process and resource manager described above can be extended and augmented by providing one or both of the following functionalities:

4.1 Multilevel Scheduling

Each process has a fixed priority, represented by a positive integer. The RL is extended to segregate processes into FIFO lists according to priorities. The following diagram shows a RL with 3 priority levels. The lowest level, 0, contains 1 process. Priority level 1 is empty. The highest priority level, 2, contains 3 processes. Each list element contains the PCB index of the corresponding process.



To support multiple priorities, the following extensions are necessary:

- The PCBs:
    - A new field, named *priority*, is included in each PCB. Assuming 3 priority levels, the possible values of the new field are 0, 1, or 2.
- Changes to functions
    - The *create* function must accept a priority value, p, as an argument: *create(p).* The value p is stored in the priority field of the new PCB.
    - The *create* function must call the *scheduler* at the end. This causes a context switch if the newly created process has a higher priority than the current process.
    - The *release* function must also call the scheduler at the end. This causes a context switch if the *release* operation reactivates a blocked process that has a higher priority than the current process.
    - The *delete* function must also call the scheduler at the end. This causes a context switch in the following situation: one of the deleted processes, j, releases a resource on which another process, k, is

blocked. If k has a higher priority than the current process, i, then k must continue running instead of i.

- The *scheduler* is extended to deal with the multilevel RL.

```
scheduler()
  find highest priority ready process j
  display: "process j running"
```

  - Starting with the highest priority level, 2, the *scheduler* finds the first non-empty list. The head of the list is the highest priority ready process j.
  - The function displays the index of the process chosen to run.

- Changes to shell

  - The command cr must accept a priority value, p, as a parameter: "cr p"
  - The possible error checks must be extended to reflect the above change, that is, the value of p must be within the defined range.

4.2. Multiunit Resources

The concept of a resource is extended to include multiple identical units. That is, each RCB represents a *class* of resources. With 4 resources, assume that:

- Resources 0 and 1 have 1 unit each
- Resource 2 has 2 units
- Resource 3 has 3 units.

To support multiple-unit resources, the following extensions are necessary:

- Changes to RCBs

  - A new field, *inventory*, is introduced in each RCB to indicate the initial number of units of the resource (1, 2, or 3).
  - The *state* field must be extended from a binary value to an integer counter, which specifies the number of currently available units.
  - Each element on the waitlist must include, in addition to the index of the blocked process, the number of units requested by the process.

- Changes to functions

  - The *request* function must accept the resource index, r, and the number of requested units, k, as arguments: *request(r, k).*
  - The *release* function must accept the resource index, r, and the number of released units, k, as arguments: *release(r, k).*
  - The *request* function must check whether a sufficient number of units is available when making the decision to allocate or to block the process. (Partial allocation of units should not be supported.)
  - The *release* function must execute a loop to unblock potentially more than one process from the waitlist. The reason is that the release of some number of units could enable zero, one, or more processes, depending on the numbers of units requested by each process. For example, releasing 3 units of resource 3 could enable up to 3 blocked processes if each requested only a single unit.

- Changes to the shell

  - The commands rq and rl must accept the resource index, r, and the number of units, k, as parameters: “rq r k” and “rl r k.”
  - The error checks must be extended to reflect the above changes, including the following:

    - The number of units requested plus the number already held by the requesting process must not exceed the initial inventory of the resource. Such a request could never be satisfied and would block the process forever.
    - The number of units released must not exceed the number of units currently held.

5. Summary of Specific Tasks

1. Design and implement the process and resource manager, which includes: The specific forms and implementation of the data structures and functions depend on which, if any, of the extensions are to be included.

    - The PCB, RCB, and RL data structures
    - The functions *create(), destroy(), request()*, *release()*, *timeout(), scheduler()*, and *init()*

2. Design and implement the shell to be able test and demonstrate the functionality of the project.
3. Instantiate the manager to include the following at start-up:

    - A process descriptor array PCB[16].
    - A resource descriptor array RCB[4].
    - A Ready List RL. If multilevel scheduling is implemented, the RL consists of 3 priority levels, 0 through 2.

4. Assume that any test sequence starts with the command “in”. The corresponding *init* function initializes all data structures and creates a single running process, PCB[0], at the lowest-priority level 0.

5. Test the manager using a variety of command sequences to explore all aspects of its behavior, including the detection of errors.

PREVIOUS (/WEBCOM-VIEW-PAGE/5/74302/74291?PAGE-ID=4829625)
NEXT (/WEBCOM-VIEW-PAGE/5/74302/74291?PAGE-ID=4829629)

For questions or concerns regarding this online publication, please contact web support (/eform/submit/support-form).

If you are having troubles viewing something on the page, make sure you have the correct plug-ins (/systemRequirements).

GRLContent™ is a trademark of Great River Learning. All rights reserved. © 2002-2026.

View the Great River Learning Privacy Statement (/showPrivacyPolicy) | Terms and Conditions (/termsOfUsePage).